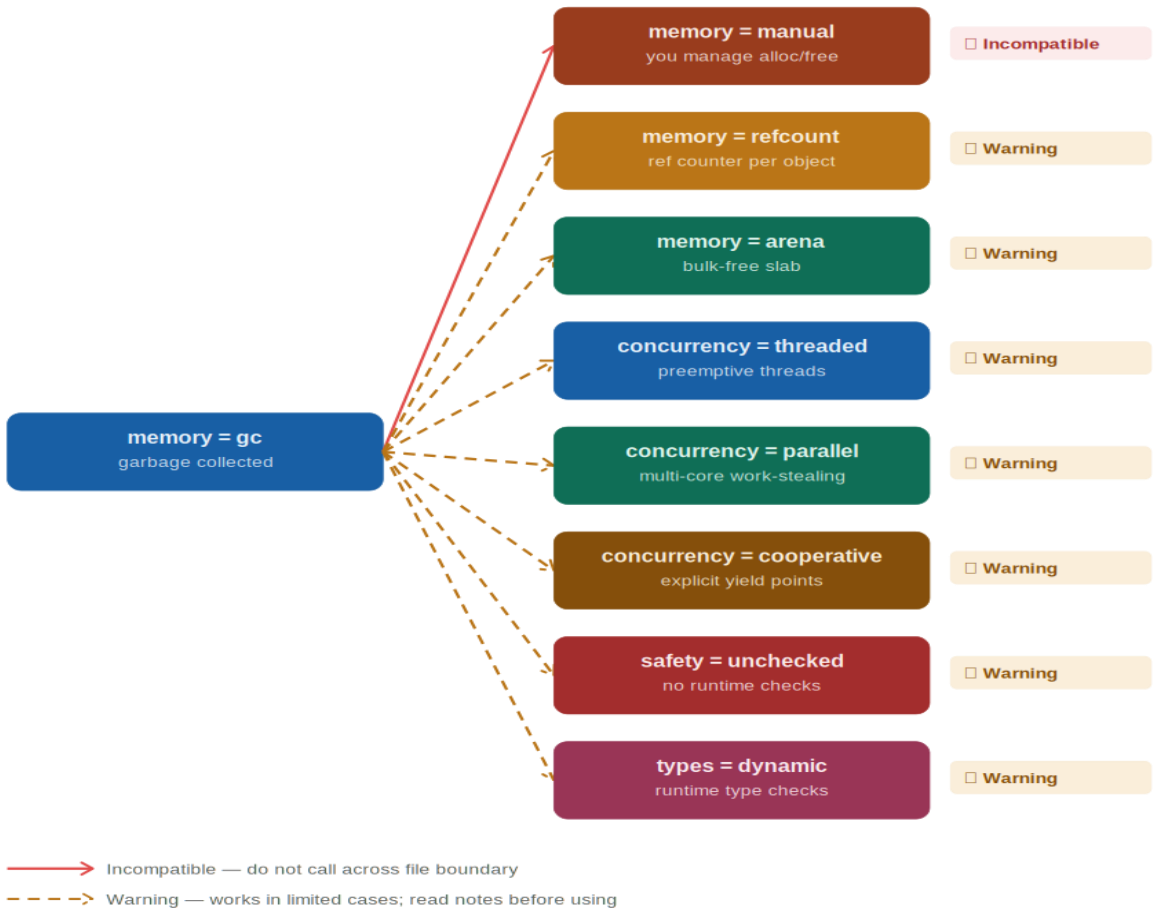
UniLogic — DR Visual Conflict Reference

Each diagram shows one setting value and every other value it conflicts with, across all settings.

How to read these diagrams: the source value is on the left. Each line connects it to a value it conflicts with — in any setting, not just its own group. A solid red line means the two values must never appear on opposite sides of a cross-file call. A dashed amber line means they can coexist with care, but there are documented traps.

Memory Management

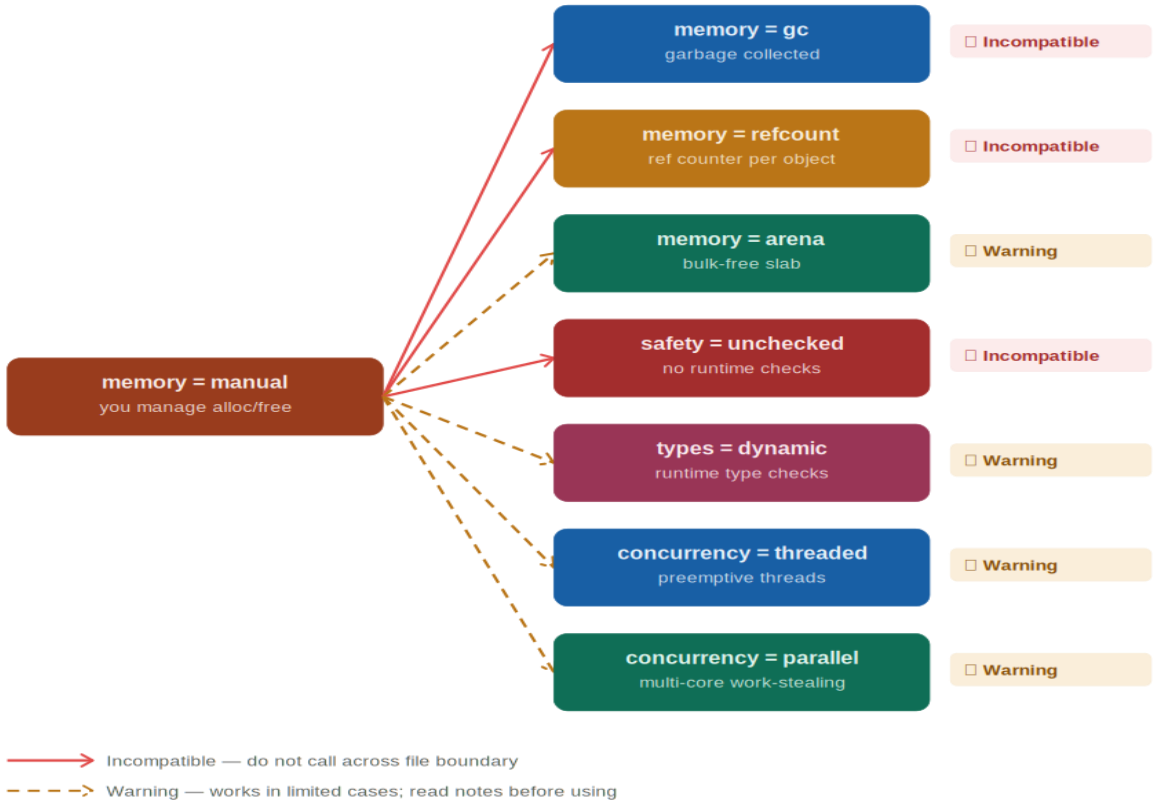
memory = gc



Conflicts with	Severity	What happens
`memory = manual`	✗ Incompatible	GC frees object; manual code holds dangling pointer → crash or silent corruption

`memory = refcount`	⚠ Warning	GC may collect before refcount reaches 0; use explicit bridges between files
`memory = arena`	⚠ Warning	Arena freed at end of scope; GC objects may outlive arena-owned data
`concurrency = threaded`	⚠ Warning	GC stop-the-world pauses block all threads; mutexes and timing behave unpredictably during collection
`concurrency = parallel`	⚠ Warning	Parallel workers stall mid-task during GC pause with no warning or coordination
`concurrency = cooperative`	⚠ Warning	GC may interrupt between yield points; whether this breaks depends on the GC implementation — compiler cannot guarantee safety
`safety = unchecked`	⚠ Warning	GC normally catches use-after-free; unchecked removes that net — missed pointer becomes silent corruption
`types = dynamic`	⚠ Warning	Dynamic types generate more short-lived heap objects, increasing GC pressure and collection frequency

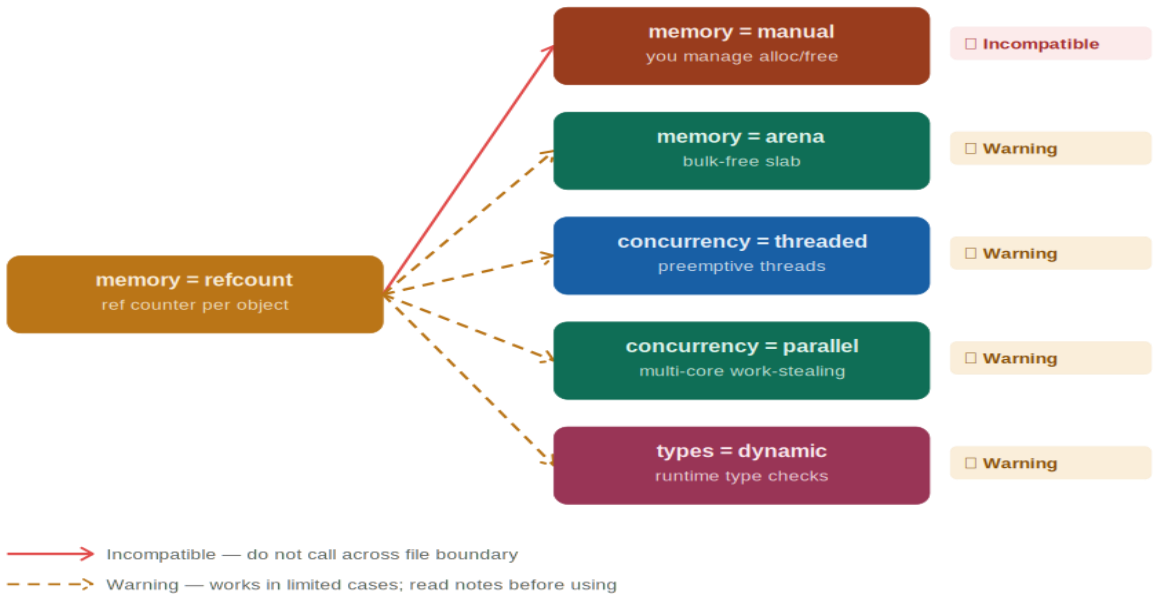
memory = manual



Conflicts with	Severity	What happens
`memory = gc`	✗ Incompatible	Manual free + GC collect same object → double-free, heap corruption
`memory = refcount`	✗ Incompatible	Both systems attempt to free the same allocation → double-free

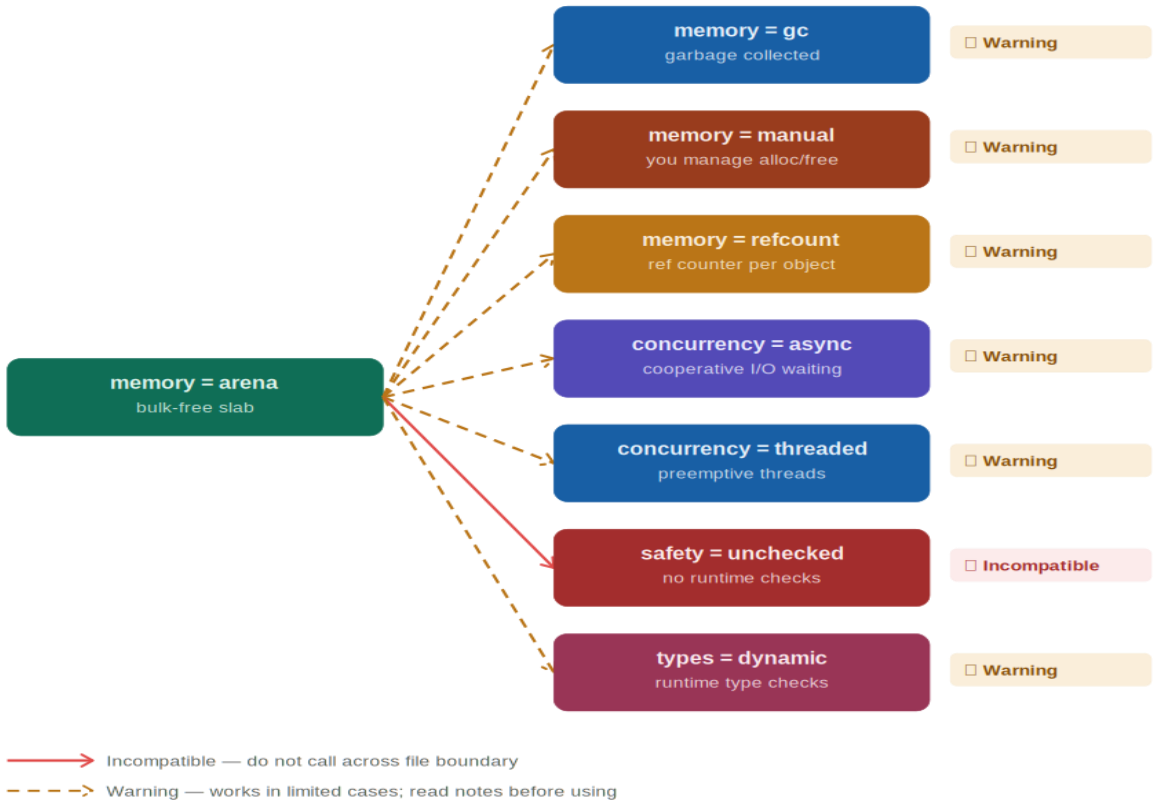
`memory = arena`	⚠ Warning	Manual code may free arena-owned memory before the arena scope ends
`safety = unchecked`	✗ Incompatible	Manual memory with no bounds checking: buffer overflows and use-after-free with zero detection — the classic exploit recipe
`types = dynamic`	⚠ Warning	Dynamic typing obscures object lifetimes; hard to know when it is safe to free an object
`concurrency = threaded`	⚠ Warning	Manual free from one thread while another thread reads same address; classic heap data race
`concurrency = parallel`	⚠ Warning	Parallel workers may all touch the same allocation; free from one worker corrupts others

memory = refcount



Conflicts with	Severity	What happens
`memory = manual`	✗ Incompatible	Refcount hits 0, memory freed; manual code still holds a pointer → use-after-free
`memory = arena`	⚠ Warning	Arena bulk-freed while refcount > 0; dangling pointer into freed slab
`concurrency = threaded`	⚠ Warning	Counter increment/decrement from multiple threads requires atomics; non-atomic refcount + threads = counter corruption
`concurrency = parallel`	⚠ Warning	Same as threaded; parallel workers race on the counter without atomic operations
`types = dynamic`	⚠ Warning	Dynamic types make accidental circular references easy; refcount can never free a cycle

memory = arena



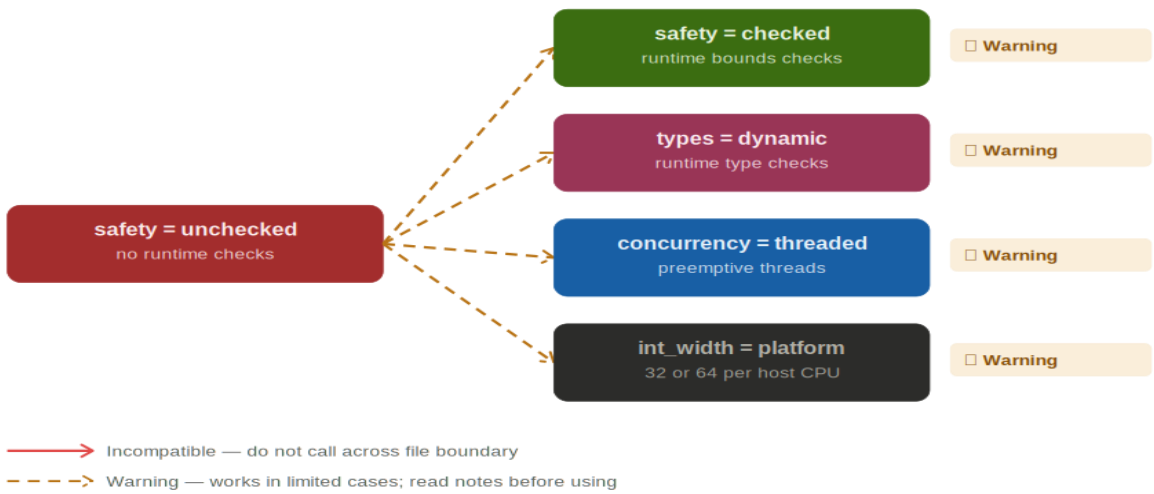
Conflicts with	Severity	What happens
`memory = gc`	⚠ Warning	GC allocations inside arena scope; lifetimes don't align with bulk-free
`memory = manual`	⚠ Warning	Arena bulk-free pulls out memory manual code still holds
`memory = refcount`	⚠ Warning	Arena outlasts refcount object; refcount freed data still accessible via arena pointer
`concurrency = async`	⚠ Warning	Async tasks may outlive the arena scope; callback fires after arena is freed → garbage read
`concurrency = threaded`	⚠ Warning	Arena is not thread-safe; concurrent allocations from multiple threads corrupt the slab pointer
`safety = unchecked`	✗ Incompatible	Arena overflow writes past slab boundary silently; overwrites adjacent memory with no error raised
`types = dynamic`	⚠ Warning	Dynamic types have variable sizes that can fragment or overflow the arena unpredictably

Safety Checking

safety = checked

Conflicts with	Severity	What happens
`safety = unchecked`	⚠ Warning	Checked code calls unchecked; errors in callee are invisible to caller's safety net

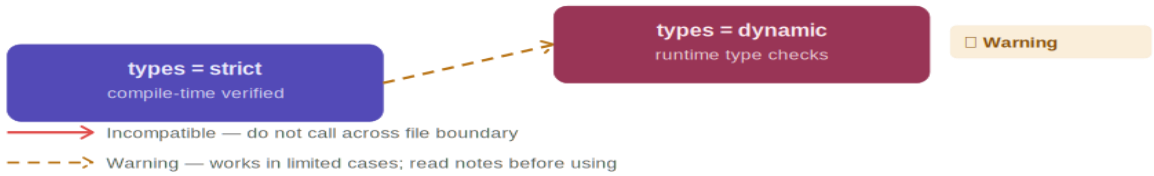
safety = unchecked



Conflicts with	Severity	What happens
`safety = checked`	⚠ Warning	Unchecked code may pass corrupted state into checked callee; error surfaces far from source
`types = dynamic`	⚠ Warning	Type mismatches that checked mode catches at runtime become silent wrong reads
`concurrency = threaded`	⚠ Warning	Data races that checked mode surfaces become invisible; wrong values with no error
`int_width = platform`	⚠ Warning	Platform-width overflow is already hard to reason about; removing overflow detection makes it invisible

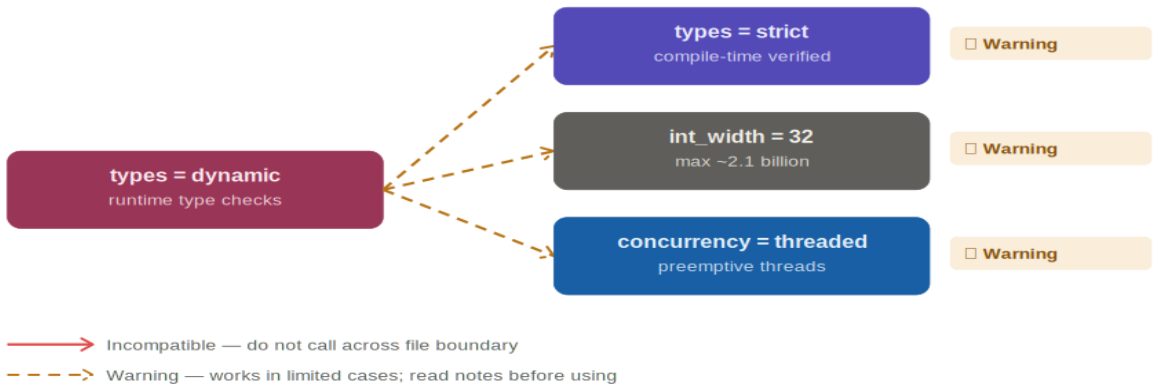
Type System

types = strict



Conflicts with	Severity	What happens
`types = dynamic`	⚠ Warning	Strict caller receives dynamic return value; type mismatch not caught until runtime

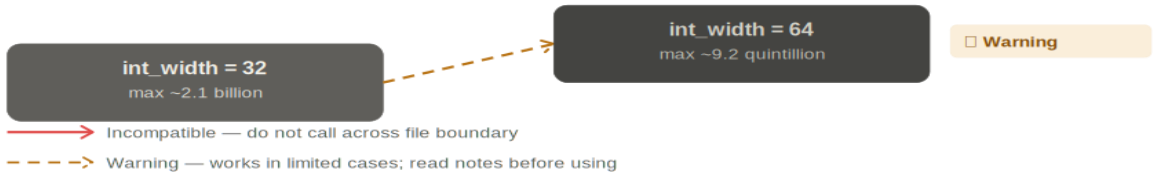
types = dynamic



Conflicts with	Severity	What happens
`types = strict`	⚠ Warning	Dynamic code passes wrong type into strict callee; crash at call boundary
`int_width = 32`	⚠ Warning	Dynamic code passing large numbers into 32-bit slots silently truncates — already in matrix, shown here for completeness
`concurrency = threaded`	⚠ Warning	Type of a shared variable can change between threads; one thread sees int, another sees string, neither errors

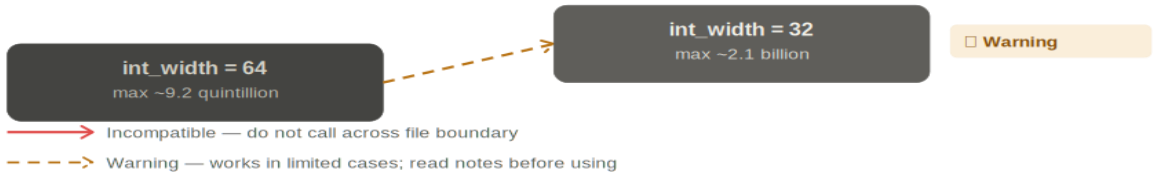
Integer Width

int_width = 32



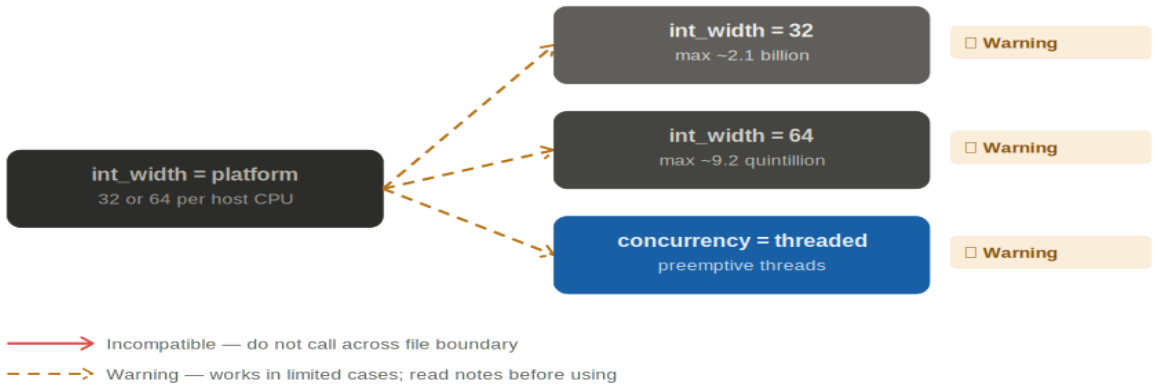
Conflicts with	Severity	What happens
`int_width = 64`	⚠ Warning	32-bit value promoted to 64-bit; generally safe but sign-extension can corrupt negatives

int_width = 64



Conflicts with	Severity	What happens
`int_width = 32`	⚠ Warning	64-bit value passed to 32-bit slot; upper 4 bytes silently discarded → wrong result

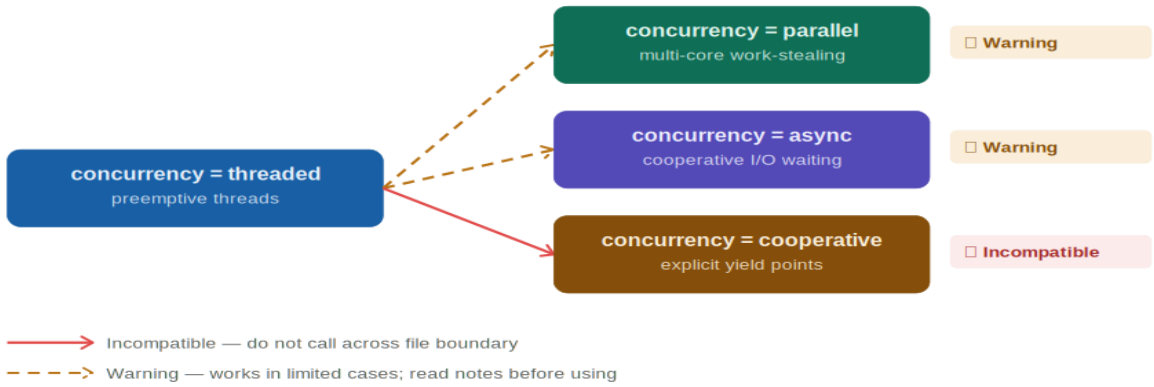
int_width = platform



Conflicts with	Severity	What happens
`int_width = 32`	⚠ Warning	Platform=64 on dev machine, 32 in production; overflow on deploy with no dev-side warning
`int_width = 64`	⚠ Warning	Platform=32 on embedded target; 64-bit callee sees truncated values
`concurrency = threaded`	⚠ Warning	Even checked mode cannot protect against platform-width race conditions on 32-bit vs 64-bit systems

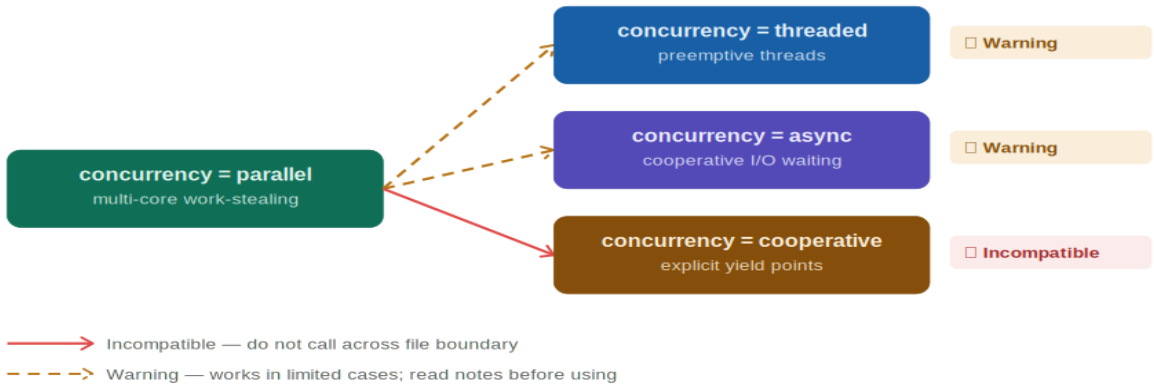
Concurrency Model

concurrency = threaded



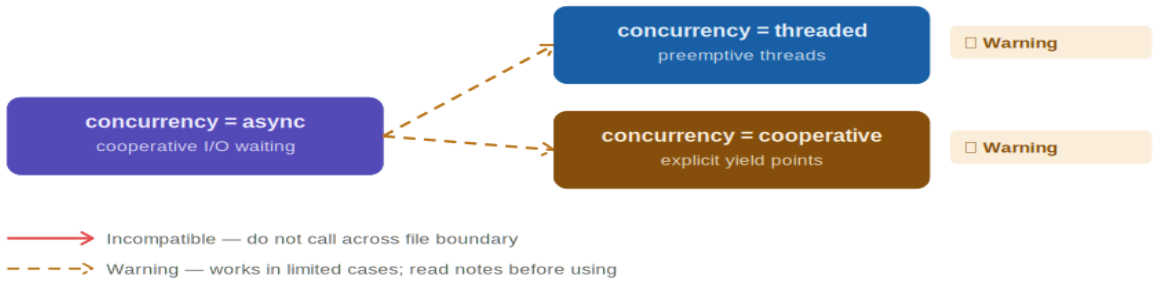
Conflicts with	Severity	What happens
`concurrency = parallel`	⚠ Warning	Thread scheduler and parallel work-stealer compete; timing-sensitive code may misbehave
`concurrency = async`	⚠ Warning	Async callbacks invoked from wrong thread context; data race possible
`concurrency = cooperative`	✗ Incompatible	Preemptive thread interrupts cooperative scheduler mid-yield; invariants broken

concurrency = parallel



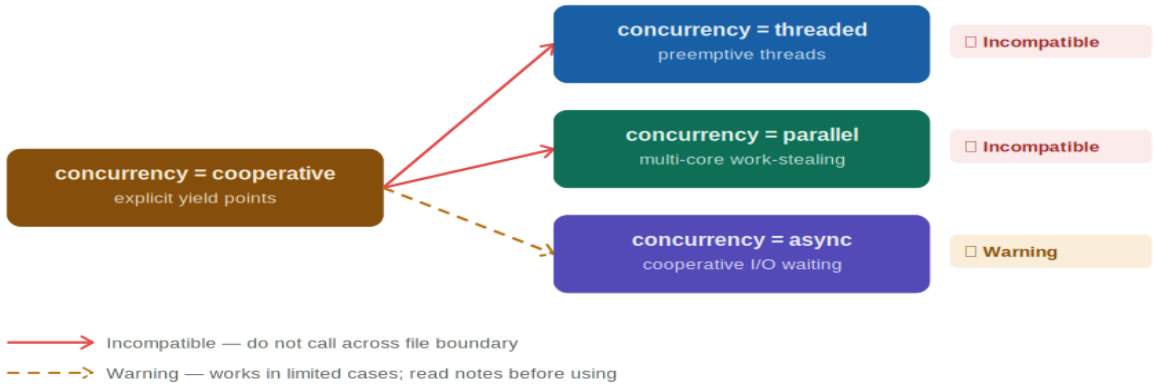
Conflicts with	Severity	What happens
`concurrency = threaded`	⚠ Warning	Parallel workers preempt; thread-local state assumptions may not hold
`concurrency = async`	⚠ Warning	Parallel dispatch fires async callbacks on arbitrary workers; ordering undefined
`concurrency = cooperative`	✗ Incompatible	Work-stealing never yields; cooperative scheduler starved permanently

concurrency = async



Conflicts with	Severity	What happens
`concurrency = threaded`	⚠ Warning	Async resumes on unpredictable thread; shared state needs explicit synchronization
`concurrency = cooperative`	⚠ Warning	Async I/O completion may resume inside cooperative turn; yield point semantics unclear

concurrency = cooperative



Conflicts with	Severity	What happens
`concurrency = threaded`	✗ Incompatible	Cooperative code never yields to thread preemptor; deadlock or starvation likely
`concurrency = parallel`	✗ Incompatible	Parallel workers don't yield; cooperative scheduler never gets control back
`concurrency = async`	⚠ Warning	Async event loop may fire before cooperative task yields; ordering not guaranteed